

ASYMPTOTIC COMPLEXITY ANALYSIS

Detailed Solutions for GATE & PSU Level Questions

Format: Step-by-Step Revision & Study Solutions Guide

Q1 (GATE 2021 Style)

Given:

$$T(n) = T(n/2) + T(2n/5) + 7n$$

$$T(0) = 1$$

Determine the asymptotic complexity of $T(n)$.

SOLUTION:

Let's solve this using the **Recursion Tree Method** (or the Akra-Bazzi framework for linear recurrences with unequal subproblems).

- **Root Level Work:** The non-recursive cost at the root is $7n$.
- **Level 1 Work:** The sub-problems split into sizes $n/2$ and $2n/5$.
Total work at Level 1 = $7(n/2) + 7(2n/5) = 7n * (1/2 + 2/5) = 7n * (9/10)$.
- **Level 2 Work:** Expanding these sub-problems further gives:
Total work at Level 2 = $7(n/4) + 7(n/5) + 7(n/5) + 7(4n/25) = 7n * (9/10)^2$.
- **General Level k Work:** At any depth k , the total work done across the level is $7n * (9/10)^k$.

To find the total time complexity, we sum up the work done across all levels of the tree:

$$T(n) = 7n + 7n(9/10) + 7n(9/10)^2 + 7n(9/10)^3 + \dots = 7n * \sum_{k=0}^{\infty} (9/10)^k$$

This forms an infinite decreasing geometric series with a common ratio $r = 9/10$. Since $r < 1$, the infinite sum converges cleanly:

$$T(n) = 7n * [1 / (1 - 9/10)] = 7n * [1 / (1/10)] = 7n * 10 = 70n$$

Since constants are dropped in asymptotic analysis, $70n$ simplifies directly to linear growth.

Final Answer:

The tight asymptotic complexity is $\Theta(n)$.

Q2 (GATE 2021 Style)

For constants $a \geq 1$ and $b > 1$:

$$T(n) = aT(n/b) + f(n)$$

Under what conditions will $T(n) = \Theta(n \log n)$ hold?

SOLUTION:

We match this expression directly against the standard **Master Theorem** for divide-and-conquer recurrences:

$$T(n) = aT(n/b) + \Theta(n^{\log_b a} \log^k n) \rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

This is Case 2 of the Master Theorem. We want the result to be $T(n) = \Theta(n \log n)$, which is equivalent to $\Theta(n^1 \log^1 n)$. Let's match the exponents:

- The exponent of n must be 1: $\log_b a = 1 \rightarrow a = b$.
- The exponent of $\log n$ must be 1: $k + 1 = 1 \rightarrow k = 0$.

Substituting $\log_b a = 1$ and $k = 0$ back into the required form of $f(n)$:

$$f(n) = \Theta(n^{\log_b a} \log^k n) = \Theta(n^1 \log^0 n) = \Theta(n)$$

Final Answer:

The required conditions are $a = b$ and $f(n) = \Theta(n)$.

Q3 (GATE 2024 Style)

An algorithm checks whether an array of size N is sorted by making a single pass and comparing only adjacent elements. What is the tightest asymptotic running time?

SOLUTION:

Let's look at how the verification process works step-by-step:

- To check if an array is sorted, the loop travels from index 0 sequentially up to index $N-2$.
- In each individual iteration, it performs exactly one constant-time comparison: checking if $A[i] \leq A[i+1]$.
- The total number of comparisons performed over a single full pass is exactly $N - 1$.
- Each comparison step along with loop management takes constant time, $\Theta(1)$.

Thus, the total running time is given by a linear function: $T(N) = c * (N - 1) = \Theta(N)$. Because the algorithm must examine the array elements sequentially, the lower and upper bounds match exactly.

Final Answer:

The tightest asymptotic running time is $\Theta(N)$.

Q4 (GATE 2015 Style)

Evaluate the asymptotic order of:

$$1^3 + 2^3 + 3^3 + \dots + n^3$$

Which of the following are true?

- $\Theta(n^4)$ • $\Theta(n^5)$ • $O(n^5)$ • $\Omega(n^3)$

Choose all correct options.

SOLUTION:

First, let's find the exact expression using the standard mathematical identity for the sum of the cubes of the first n natural numbers:

$$\sum_{i=1}^n i^3 = [(n(n+1)) / 2]^2$$

Expanding the algebraic terms:

$$[(n^2 + n) / 2]^2 = (n^4 + 2n^3 + n^2) / 4$$

The highest degree term in the resulting polynomial is n^4 , meaning the exact tight asymptotic order is $\Theta(n^4)$. Let's evaluate each option based on asymptotic definitions:

- $\Theta(n^4)$: **True**, since the highest degree term matches exactly.
- $\Theta(n^5)$: **False**, a degree-4 polynomial doesn't grow at the same rate as a degree-5 term.
- $O(n^5)$: **True**, because Big-O represents an upper bound. Since $n^4 \leq c * n^5$ for all large n , this is mathematically correct.
- $\Omega(n^3)$: **True**, because Big-Omega represents a lower bound. Since $n^4 \geq c * n^3$ for all large n , this is also correct.

Final Answer:

The correct options are $\Theta(n^4)$, $O(n^5)$, and $\Omega(n^3)$.

Q5 (GATE 2015 Style)

```
int fun1(int n)
{
    int i, j, k, p, q=0;

    for(i=1; i<n; ++i)
    {
        p=0;

        for(j=n; j>1; j=j/2)
            ++p;

        for(k=1; k<p; k=k*2)
            ++q;
    }

    return q;
}
```

Find the asymptotic value of q .

SOLUTION:

Let's trace how the nested loop structure behaves step-by-step:

- **Outer Loop:** ``for(i=1; i<n; ++i)`` runs exactly **$n-1$** times. This gives an outer loop complexity of **$\Theta(n)$** .
- **First Inner Loop (computing p):** ``for(j=n; j>1; j=j/2) ++p;``
The loop control variable **j** starts at **n** and is repeatedly halved until it drops to 1. The total number of divisions performed is **$\lfloor \log_2 n \rfloor$** . Therefore, inside each outer loop iteration, the variable **p** becomes exactly **$\lfloor \log_2 n \rfloor$** .
- **Second Inner Loop (incrementing q):** ``for(k=1; k<p; k=k*2) ++q;``
Here, **k** starts at 1 and is repeatedly multiplied by 2 until it reaches or exceeds **p** . This is a classic logarithmic loop that runs **$\lfloor \log_2 p \rfloor$** times.
Plugging in the value of **p** from the step above, the number of executions per outer loop iteration is **$\log_2 (\log_2 n)$** .

Since the second inner loop increments **q** exactly **$\log_2 (\log_2 n)$** times during each of the **$n-1$** outer iterations, we multiply them to find the overall complexity:

$$\text{Total Increments of } q = (n - 1) * \log_2 (\log_2 n) = \Theta(n \log \log n)$$

Final Answer:

The asymptotic value of **q** is **$\Theta(n \log \log n)$** .

Q6 (Classic GATE)

```
for(int i=n; i>0; i/=2)
{
    for(int j=0; j<i; j++)
    {
        // constant work
    }
}
```

Determine the complexity.

SOLUTION:

Let's count how many times the constant work inside the inner loop runs as the outer loop index **i** decreases:

- 1st iteration of outer loop: **$i = n$** → Inner loop runs **n** times.
- 2nd iteration of outer loop: **$i = n/2$** → Inner loop runs **$n/2$** times.
- 3rd iteration of outer loop: **$i = n/4$** → Inner loop runs **$n/4$** times.
- This sequence continues down until **i** drops to 1.

Summing up the total work done across all iterations:

$$\text{Total Work} = n + n/2 + n/4 + n/8 + \dots + 1$$

Factoring out n from the expression:

$$\text{Total Work} = n * (1 + 1/2 + 1/4 + 1/8 + \dots)$$

The term inside the brackets is a standard decreasing geometric series. Even extended to infinity, it sums to exactly 2 ($\sum_{k=0}^{\infty} (1/2)^k = 2$). Therefore:

$$\text{Total Work} < 2n = \Theta(n)$$

Final Answer:

The complexity is $\Theta(n)$.

Q7 (Classic GATE)

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=i;j++)
    {
        // constant work
    }
}
```

Determine the complexity.

SOLUTION:

Let's check how many times the inner loop runs for each sequential step of the outer loop:

- For $i = 1$, inner loop runs 1 time.
- For $i = 2$, inner loop runs 2 times.
- For $i = 3$, inner loop runs 3 times.
- ... For $i = n$, inner loop runs n times.

Summing up the operations yields the sum of the first n natural numbers:

$$\text{Total Work} = 1 + 2 + 3 + \dots + n = [n(n+1)] / 2 = (n^2 + n) / 2$$

Dropping the lower-order term (n) and the constant divisor ($1/2$), we get the dominant term.

Final Answer:

The complexity is $\Theta(n^2)$.

Q8 (Recursion)

```
void fun(int n)
{
    if(n<=1)
        return;

    fun(n/2);
    fun(n/2);
}
```

Determine the complexity.

SOLUTION:

Let's write down the recurrence relation for this recursive structure. Each function call performs a constant-time check ($\Theta(1)$) and triggers two identical recursive calls, each dealing with half the input size:

$$T(n) = 2T(n/2) + c$$

We apply the standard **Master Theorem**: $a = 2$, $b = 2$, and $f(n) = c = \Theta(n^0)$.

- Find the critical term: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
- Compare $f(n) = \Theta(n^0)$ with n^1 .
- Since $f(n) = O(n^{1-\epsilon})$ for $\epsilon = 1$, this falls under **Case 1** of the Master Theorem.

According to Case 1, the overall running time is bounded by the total work done at the leaf nodes:

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n)$$

Final Answer:

The complexity is $\Theta(n)$.

Q9 (Fibonacci Pattern)

```
int fib(int n)
{
    if(n<=1)
        return n;

    return fib(n-1)+fib(n-2);
}
```

Determine the complexity.

SOLUTION:

The recurrence relation representing this naive recursive implementation of Fibonacci numbers is:

$$T(n) = T(n-1) + T(n-2) + c$$

Let's analyze bounds by looking at a simplified recursion tree:

- **Upper Bound:** If we assume both recursive branches take the maximum size $n-1$, we get $T(n) \leq 2T(n-1) + c$, which solves to $O(2^n)$.
- **Lower Bound:** If we assume both branches take the minimum size $n-2$, we get $T(n) \geq 2T(n-2) + c$, which solves to $\Omega(2^{n/2}) = \Omega(1.414^n)$.

Solving the exact characteristic equation ($r^2 - r - 1 = 0$) yields the root corresponding to the golden ratio $\phi = (1 + \sqrt{5})/2 \approx 1.618$. Therefore, the precise tight bound is:

$$T(n) = \Theta(\phi^n) = \Theta(1.618^n)$$

This represents clear exponential growth.

Final Answer:

The exact tight asymptotic complexity is $\Theta(1.618^n)$ (often loosely bounded as $O(2^n)$).

Q10 (GATE Favorite)

Arrange the following functions in increasing order of their asymptotic growth:

$$n^{3/2}, \quad n \log n, \quad n^{\log_2 n}, \quad 2^n$$

SOLUTION:

Let's compare the growth rates of these functions as n grows arbitrarily large:

- $n \log n$: Linearithmic growth. This grows slightly faster than linear (n) but slower than any polynomial with a power greater than 1.
- $n^{3/2} = n^{1.5}$: Polynomial growth. Since $n^{0.5}$ eventually beats $\log n$, we know that $n \log n < n^{1.5}$.
- $n^{\log_2 n}$: Quasi-polynomial growth. We can rewrite this with base 2 as $2^{(\log_2 n)^2}$. As $n \rightarrow \infty$, $(\log_2 n)^2$ grows much faster than a constant power like 1.5, making this larger than $n^{1.5}$.
- 2^n : Exponential growth. Comparing exponents in base 2, n grows much faster than $(\log_2 n)^2$, meaning 2^n has the highest growth rate here.

Final Answer:

The correct increasing order of growth is:

$$n \log n < n^{3/2} < n^{\log_2 n} < 2^n$$

ADDITIONAL PSU-LEVEL QUESTIONS

Q11

Determine the complexity of: $T(n) = T(n-1) + 1$

SOLUTION:

Let's unfold the recurrence equation step-by-step:

$$T(n) = T(n-1) + 1$$

Substitute the definition for $T(n-1)$:

$$T(n) = [T(n-2) + 1] + 1 = T(n-2) + 2$$

Unfolding it a third time gives:

$$T(n) = T(n-3) + 3$$

Generalizing this pattern for k substitutions:

$$T(n) = T(n-k) + k$$

To hit the base case, we set $n - k = 0 \rightarrow k = n$:

$$T(n) = T(0) + n = c + n = \Theta(n)$$

Final Answer:

The complexity is $\Theta(n)$.

Q12

Determine the complexity of: $T(n) = T(n-1) + n$

SOLUTION:

Let's expand this recurrence using sequential substitution:

$$T(n) = T(n-1) + n$$

$$T(n) = [T(n-2) + (n-1)] + n = T(n-2) + (n-1) + n$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

Continuing this expanding chain down to the base case $T(0)$:

$$T(n) = T(0) + 1 + 2 + 3 + \dots + (n-1) + n$$

The sum of the first n positive integers is given by $[n(n+1)] / 2$:

$$T(n) = c + (n^2 + n)/2 = \Theta(n^2)$$

Final Answer:

The complexity is $\Theta(n^2)$.

Q13

Determine the complexity of: $T(n) = 2T(n-1) + 1$

SOLUTION:

Let's substitute the relation successively:

$$T(n) = 2T(n-1) + 1$$

$$T(n) = 2[2T(n-2) + 1] + 1 = 4T(n-2) + 2 + 1$$

$$T(n) = 4[2T(n-3) + 1] + 2 + 1 = 8T(n-3) + 4 + 2 + 1$$

Generalizing this for k total steps:

$$T(n) = 2^k T(n-k) + (2^{k-1} + 2^{k-2} + \dots + 2^0)$$

Setting $k = n$ to reach our base case boundary $T(0)$:

$$T(n) = 2^n T(0) + (2^n - 1) = (c + 1)2^n - 1 = \Theta(2^n)$$

Final Answer:

The complexity is $\Theta(2^n)$.

Q14

Determine the complexity of: $T(n) = T(n/2) + 1$

SOLUTION:

We solve this using the standard **Master Theorem** framework:

$$T(n) = aT(n/b) + f(n)$$

For this query, the parameters are $a = 1$, $b = 2$, and the driving work function is $f(n) = 1 = \Theta(n^0)$.

- Find the baseline exponent: $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$.
- Compare $f(n) = \Theta(n^0)$ with n^0 . They are equal.
- This satisfies **Case 2** of the Master Theorem with a factor of $k = 0$.

The Case 2 formula yields: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$. Substituting our values:

$$T(n) = \Theta(n^0 \log^1 n) = \Theta(\log n)$$

Final Answer:

The complexity is $\Theta(\log n)$.

Q15

```
for(int i=1;i<=n;i*=2)
{
    for(int j=1;j<=n;j++)
    {
    }
}
```

Determine the complexity.

SOLUTION:

Since the two loops are nested and their execution limits do not depend on each other's current values, we can evaluate their iteration counts separately:

- **Outer Loop:** `for(int i=1; i<=n; i\*=2)`

The index variable doubles with each loop cycle: $i = 1, 2, 4, 8, \dots, \leq n$. This is a classic logarithmic progression. The total number of steps is $\lfloor \log_2 n \rfloor + 1$, which simplifies to $\Theta(\log n)$ iterations.

- **Inner Loop:** `for(int j=1; j<=n; j++)`

This loop increments sequentially from 1 up to n . It performs exactly n steps for every single iteration of the outer loop, which is $\Theta(n)$ work.

To find the final complexity, we multiply the outer iteration count by the inner loop work:

$$\text{Total Time Complexity} = (\text{Outer iterations}) * (\text{Inner work}) = \Theta(\log n) * n = \Theta(n \log n)$$

Final Answer:

The complexity is $\Theta(n \log n)$.